

Устройство для считывания штрихкодов на конвейере

Технический отчёт и архитектура базового прототипа

Исходные условия: конвейер 650 мм; скорость 1 м/с; номинальный интервал между коробками 2 с; коробка 600×400×400 мм; несколько штрихкодов на коробке; штрихкод может находиться на любой видимой грани; расстояние до сортировочной секции — не более 3 м.

Ключевой вывод: для этой задачи рациональна не Vision Transformer-система «сразу читает код», а двухступенчатая схема: быстрый детектор штрихкодов (YOLO) → вырезание/нормализация ROI → специализированный barcode decoder. Это разделяет поиск объекта и чтение payload, снижает вычислительную нагрузку и упрощает валидацию.

1. Предлагаемая аппаратная архитектура

Базовый вариант — пять камер: одна сверху и четыре вокруг вертикальных граней. Камеры должны смотреть преимущественно перпендикулярно соответствующей грани, что уменьшает перспективные искажения и повышает вероятность декодирования.

Компонент	Рекомендация	Обоснование
Камеры	5× Basler ace 2 R Pro a2A2448-23gmPRO, Mono, Global Shutter, 2448×2048, 23 fps, GigE	5 MP, global shutter, промышленный GigE-интерфейс. Basler указывает Sony IMX547, global shutter, 5 MP и 23 fps. [cite]turn0search5
Освещение	5 зон pulsed LED, предпочтительно с рассеивателем; при бликах — поляризация	Короткий строб позволяет держать экспозицию порядка сотен микросекунд и уменьшить motion blur при 1 м/с.
Триггер	Промышленный photoelectric sensor перед зоной камер	Даёт детерминированный момент начала окна съёмки; конкретный SICK W4S-3 является подходящим классом датчиков с промышленными IP-классами. [cite]turn0search48
Вычислитель	NVIDIA Jetson Orin NX 16 GB + NVMe	Локальная GPU-обработка, достаточно запаса для нескольких потоков CV и TensorRT. Линейка Orin NX поддерживает до 157 TOPS для 16 GB-версии в зависимости от режима/метрики NVIDIA. [cite]turn0search6 [turn0search9]
Сеть	Industrial Ethernet; отдельный switch для камер; uplink к PLC/WCS	GigE Vision хорошо подходит для нескольких камер; Basler pylon предоставляет Python/C++ API и средства работы с GigE Vision. [cite]turn1search0 [turn1search8]

Физическое расположение

Камеры образуют короткий inspection tunnel. Верхняя камера смотрит вниз; две боковые — на левую и правую грани; две торцевые — на переднюю и заднюю грани по ходу движения. Фотоэлектрический датчик располагается upstream. Между датчиком и камерной зоной задаётся фиксированное расстояние, а задержка триггера вычисляется как $t = d / v$.

Важное ограничение: если под «любой гранью» имеется в виду также физическое дно коробки, пяти камер недостаточно. Дно закрыто конвейером. Для строгого требования нужно либо оптическое окно/зазор и нижняя камера, либо механизм приподнимания/переворота коробки. Это не программная проблема и должно быть решено в механической части.

2. Оптический расчёт и motion blur

Для выбранной 5 MP камеры горизонтальное разрешение 2448 пикселей. При FOV порядка 650–700 мм получается ориентировочно 3,5–3,8 px/mm. Например, штрихкод шириной 50 мм занимает порядка 175–190 пикселей до учёта перспективы. Это предварительный расчёт: финальная оптика должна быть пересчитана по реальному минимальному размеру штрихкода из задания/образцов.

При скорости 1 м/с перемещение за 1 мс равно 1 мм. Поэтому целевой диапазон экспозиции — примерно 0,2–0,5 мс, если позволяет освещение. В промышленной системе это проще получить pulsed LED strobe, чем повышением постоянного света.

Камеры с global shutter предпочтительнее rolling shutter для движущейся коробки; выбранная Basler-модель имеет global shutter. [cite\turn0search5]

3. Вычислительная архитектура

Обработку следует выполнять локально. Передача кадров в облако не даёт преимуществ для задачи с жёстким latency budget и создаёт зависимость от сети. На Jetson выполняются capture, inference, barcode decoding, aggregation, logging и передача результата в PLC/WCS.

Jetson Orin NX 16 GB имеет 8-ядерный Cortex-A78AE CPU и NVIDIA Ampere GPU; официальные спецификации NVIDIA указывают до 157 TOPS для Orin NX 16 GB. [cite\turn0search6]

Обучение модели выполняется отдельно на workstation/cloud GPU. Для deployment модель экспортируется в TensorRT; Ultralytics документирует экспорт в TensorRT, включая FP16 и INT8, с калибровочным датасетом для INT8. [cite\turn1search1\turn1search5]

4. Алгоритм end-to-end

1. Photoelectric sensor фиксирует вход коробки.
2. Контроллер создаёт box_id и рассчитывает момент съёмки/inspection window.
3. Пять камер получают один или несколько hardware-triggered кадров.
4. Для каждого кадра YOLO детектирует все объекты класса barcode.
5. Для каждого bbox добавляется небольшой margin и создаётся ROI.
6. ROI нормализуется: grayscale/CLAHE/threshold/sharpen/upscale; при сильной перспективе выполняется rectification.
7. Barcode decoder пытается прочесть payload; для разных symbologies допускается несколько попыток preprocessing.
8. Результаты нескольких кадров/камер объединяются по box_id, дедуплицируются и валидируются.
9. Система формирует набор кодов и отправляет его в PLC/WCS.
10. Если часть кодов не прочитана, система фиксирует incomplete/failure и сохраняет диагностические кадры.

Для decoder можно использовать ZBar/pyzbar как базовый компонент: ZBar поддерживает распространённые линейные коды и QR; pyzbar предоставляет Python-интерфейс.

[cite\turn0search0\turn0search3] В production стоит проверить поддержку именно тех symbologies, которые реально используются на складе, и при необходимости добавить второй decoder.

5. Почему YOLO, а не Vision Transformer

Для первой версии ViT не нужен. В задаче нет необходимости строить тяжёлую end-to-end модель, которая сама понимает payload. Нужно быстро найти несколько небольших прямоугольных областей на каждом кадре. Небольшой YOLO детектор хорошо соответствует этому шаблону: одна модель, один класс, TensorRT, малый latency и простой deployment.

Рекомендация: начать с YOLO26n и сравнить с YOLO26s только по данным production-like теста.

Ultralytics поддерживает fine-tuning на пользовательском датасете с переносом pretrained weights и экспорт в TensorRT. [cite\turn1search3\turn1search9]

Важно: detector не должен распознавать строку штрихкода. Его задача — найти bbox. Специализированный decoder лучше использовать для чтения payload и проверки checksum/symbology.

6. Датасет и обучение

Стартовый объём: 3 000–5 000 полноразмерных кадров для прототипа; для production — ориентир 10 000+ кадров при наличии разнообразных hard cases. Необходимо дополнительно собрать минимум около 1 000 hard-negative кадров без штрихкодов, но с текстом, логотипами, лентой, упаковочными наклейками и другими barcode-like объектами.

Разбиение: 70/15/15 train/val/test, но разделять нужно по физическим коробкам и сессиям съёмки, а не случайно по кадрам одного видео. Иначе test leakage даст завышенную оценку.

Разметка: один класс barcode, один bounding box на каждый видимый штрихкод. Payload в разметке detector не хранится.

Аугментации: brightness/contrast, blur/motion blur, JPEG noise, небольшая rotation, perspective, scale, shadows/glare, partial occlusion, sensor noise. Не стоит использовать чрезмерно агрессивные геометрические аугментации, создающие физически нереалистичные штрихкоды.

Ultralytics использует стандартный YOLO-format с нормализованными bbox; fine-tuning от pretrained weights снижает требования к объёму данных по сравнению с обучением с нуля.

`[cite]turn1search2[turn1search9]`

7. Метрики приёмки

Метрика	Что измеряет
Detection recall @ IoU 0.5	Долю видимых штрихкодов, которые нашёл detector
Decode success / barcode	Долю реально считанных штрихкодов среди видимых
All-codes-per-box success	Доля коробок, где считаны все требуемые коды — главная end-to-end метрика
False decode rate	Случаи неверно распознанного payload
Duplicate rate	Ошибочные дубли при multi-frame/multi-camera aggregation
p50/p95/p99 latency	Время от trigger до готового результата
Missed-box rate	Коробки без результата
PLC delivery success	Доля результатов, успешно принятых системой управления

8. Latency и связь с сортировкой

При длине коробки 600 мм и скорости 1 м/с время прохождения характерного размера — около 0,6 с. Интервал 2 с оставляет значительный operational margin. Тем не менее систему следует проектировать как asynchronous state machine, а не как таймер с предположением идеальных 2 секунд.

Результат должен быть готов существенно раньше сортировочной точки. Практическая цель прототипа — p99 end-to-end latency < 300 мс, после чего значение проверяется экспериментально на реальном железе. Если расстояние от центра inspection zone до sorter равно d , а скорость v , допустимый budget примерно $< d/v$ с запасом на PLC/actuator.

9. Масштабирование

Для другого размера коробок меняются FOV, рабочие расстояния, крепления и ROI. Архитектура остаётся той же. Для увеличения ширины конвейера можно использовать больше камер или камеры с большим сенсором/FOV, но необходимо сохранить минимальный pixel coverage штрихкода.

Для новых типов кодов detector остаётся тем же классом barcode, если геометрия похожа. Меняется decoder layer. Если появляются 2D-коды, QR/DataMatrix/Aztec и т.п., следует отдельно проверить decoder и, возможно, добавить второй specialized decoder.

Если геометрия штрихкодов сильно наклонена, можно перейти от обычного bbox detector к oriented bounding boxes либо добавить отдельный этап оценки quadrilateral. Делать это стоит только после анализа реальных failure cases.

10. ПО и библиотеки

Рекомендуемый стек: Python для прототипа; rpylon для камер Basler; OpenCV для image processing; Ultralytics для обучения/экспорта YOLO; TensorRT для edge inference; ZBar/pyzbar для базового decoding; PyYAML для конфигурации. Basler документирует rpylon как официальный Python binding к pylon API. [cite]turn1search6[turn1search12]

Для промышленной версии acquisition и критические части pipeline можно перенести на C++ после подтверждения алгоритма; Python оставить для tooling/diagnostics, если профилирование показывает достаточную производительность.

11. Риски и меры

Риск	Мера
Barcode на нижней грани	Нижняя камера через технологический гар или механический переворот
Motion blur	Global shutter + strobe + exposure ~0,2-0,5 ms
Блики на ламинированной этикетке	Поляризация, рассеянное освещение, несколько углов света
Малый barcode	Увеличить разрешение/FOV, изменить рабочую дистанцию, добавить ROI camera
Несколько кодов рядом	Detector one bbox per barcode + decoder per crop
Похожие надписи/логотипы	Hard-negative dataset + conservative confidence threshold
Потеря камеры/сети	Watchdog, reconnect, health status, PLC fail-safe
Неверный payload	Checksum/symbology validation + repeated-frame consensus

12. Базовый прототип

В приложенном проекте реализован reference pipeline: изображение → YOLO detection → crop → несколько вариантов preprocessing → ZBar/pyzbar decode → JSON. Также добавлены конфигурация, команды обучения/экспорта, заготовка rpylon capture и prompt для coding/research агента.

Это не следует считать промышленно готовой системой до получения production-like test set и измерения all-codes-per-box success, false decode rate и p99 latency.

13. Источники

- Basler ace 2 R Pro a2A2448-23gmPRO: характеристики 5 MP, global shutter, 2448×2048, 23 fps, Mono, GigE. [cite]turn0search5
- Basler pylon API / rpylon: официальные API и Python binding, включая работу с GigE Vision. [cite]turn1search0[turn1search6][turn1search8]
- NVIDIA Jetson Orin: спецификации Orin NX 16 GB и производительность линейки. [cite]turn0search6[turn0search9]
- Ultralytics: обучение custom dataset/fine-tuning и TensorRT export. [cite]turn1search3[turn1search9][turn1search1]
- ZBar / pyzbar: поддерживаемые типы штрихкодов и Python interface. [cite]turn0search0[turn0search3]
- SICK W4S-3: пример промышленного photoelectric sensor класса IP66/67/68/69K. [cite]turn0search48